

Análise de Código Java

LongestPalindrome — Expansão a partir do Centro

1. Objetivo Geral

A classe **LongestPalindrome** tem como finalidade encontrar a maior substring palindrômica dentro de uma string fornecida. Um palíndromo é uma sequência de caracteres que permanece igual quando lida de trás para frente (ex: "racecar", "bab"). O método principal **longestPalindrome** recebe uma string *s* e retorna a maior subsequência contígua que satisfaz essa propriedade.

2. Lógica e Algoritmo

O algoritmo adota a técnica de **expansão a partir do centro** (*expand around center*). Para cada posição *i* da string, são verificadas duas possibilidades: palíndromo de comprimento **ímpar** (centro em *i*) e de comprimento **par** (centro entre *i* e *i+1*). O método auxiliar **expand** expande os ponteiros *l* e *r* enquanto os caracteres forem iguais, retornando o comprimento do maior palíndromo encontrado. Os índices *start* e *end* são atualizados sempre que um palíndromo maior que o atual é encontrado, e ao final a substring correspondente é extraída com **substring(start, end+1)**.

3. Conceitos e Recursos Java Envolvidos

O código utiliza recursos fundamentais do Java: **métodos estáticos** (*static*) para organizar funções sem necessidade de instância; **String** e seus métodos nativos (*isEmpty*, *charAt*, *length*, *substring*); **Math.max** para comparação de valores inteiros; e estruturas de controle básicas como **for** e **while**. Não há uso de generics, threads ou coleções — a solução é deliberadamente minimalista e eficiente.

4. Complexidade de Tempo e Espaço

A complexidade de **tempo** é **$O(n^2)$** , onde *n* é o comprimento da string: para cada um dos *n* centros, a expansão pode percorrer até *n* caracteres. A complexidade de **espaço** é **$O(1)$** , pois o algoritmo utiliza apenas variáveis inteiras auxiliares, sem estruturas de dados adicionais. Existe uma solução $O(n)$ — o algoritmo de Manacher —, porém esta abordagem $O(n^2)$ é preferida pela clareza e facilidade de implementação em contextos práticos.

5. Decisões de Design e Limitações

Uma decisão relevante é a separação da lógica de expansão em um método privado (**expand**), promovendo reusabilidade e legibilidade. O código trata apenas strings com caracteres ASCII padrão; caracteres Unicode multibyte (como emojis ou certos ideogramas) podem gerar resultados incorretos, pois *charAt* opera em unidades **char** de 16 bits, não em code points completos. Além disso, quando há múltiplos palíndromos de mesmo comprimento máximo, o método retorna apenas o primeiro encontrado.

Código Analisado:

```
public class LongestPalindrome {
    public static String longestPalindrome(String s) {
        if (s.isEmpty()) return "";
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            int odd = expand(s, i, i);
            int even = expand(s, i, i + 1);
            int len = Math.max(odd, even);
            if (len > end - start) {
                start = i - (len - 1) / 2;
                end = i + len / 2;
            }
        }
        return s.substring(start, end + 1);
    }

    private static int expand(String s, int l, int r) {
        while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
            l--; r++;
        }
        return r - l - 1;
    }
}
```